

0.10.2. Tout l'alphabet, avec poserS — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 5 et 6.

1. Ce qu'il faut comprendre

C'est le 0.10 et le 0.10.1 réunis : l'alphabet du 0.10 (lignes 0 et 1), puis celui du 0.10.1 (lignes 5 et 6).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1449 octets** de cartouche.

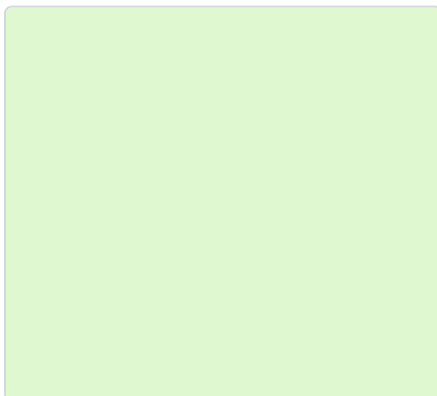
1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // 1. Lignes 0 et 1 (le 0.10)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poserS(i, 0, ALPHABET[i]); // poserS passe tout seul à la ligne 1
    }
    // 2. Lignes 5 et 6 (le 0.10.1)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poserS(i, 5, ALPHABET[i]); // poserS passe tout seul à la ligne 6
    }

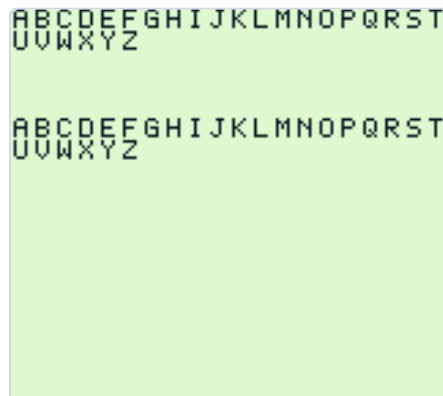
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Tout l'alphabet deux fois : en haut, et trois lignes plus bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ lignes 0 et 1

✓ lignes 5 et 6

Ensuite : **0.11. Tout l'alphabet, avec textS** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++